

통합구현

웹 애플리케이션 로그와 오류관리

2022.01.01



로그관리개요

Section 1

1.1 로그관리의 필요성

로그(Log)는 프로그램을 개발하거나 운영할 때 발생하는 문제점을 추적하거나 운영 상태를 모니터링하기 위한 텍스트 형식의 데이터이다. 콘솔에 출력되는 메시지를 기본으로 파일 및 데이터베이스 연계 등 시스템의 규모가 커질수록 더욱 체계적인 로그 관리가 요구된다.

지금 까지 여러 예제를 다루면서 프로그램 진행 과정 중 데이터베이스 연결에 성공한 경우나 웹 페이지를 전환할 때 매개변수 전달 확인, 특정 메서드 호출 확인 등이 필요한 경우에 변수 값을 출력하거나 메서드 호출 결과를 `System.out.println()` 메서드를 이용해 콘솔로 출력해 봤다. 이처럼 콘솔을 이용해 프로그램 개발이나 운영 과정에서 문제점을 추적하기 위한 메시지 출력은 가장 초보적인 로깅의 형태라고 할 수 있다.

일반적으로 로깅은 기록을 남긴다는 의미이므로, 단순한 메시지의 출력보다는 파일 형태의 기록으로 남기는 형태를 말한다. 로깅의 필요성을 정리하면 다음과 같다.

- 프로그램 개발 과정에서의 간단한 구현 검증
- 운영 시스템에서 프로그램 진행 과정의 모니터링
- 서버 기반 프로그램에서 사용자 접속 및 시스템 이용관리
- 문제 발생 시 원인 파악을 위한 시스템 분석
- 사용자 접속 및 요청 등 보안 관점에서의 데이터 관리
- 이용 형태 분석을 통한 사후 서비스 개선 목적

이 외의 다양한 목적에서 로그를 분석하며, 어떤 측면에서는 휴대폰 통화기록, SNS 메시지 기록, 쇼핑몰 쇼핑 기록 등이 모두 로그화되어 관리된다고 볼 수도 있다. 최근에는 이러한 로그 데이터들이 방대해짐에 따라 '빅데이터'라는 관점에서 다양하게 분석하고 더욱 가치 있는 정보를 추출해내는 것에 대한 관심이 높아지고 있다.

1.2 로그 유형 및 로깅 구현 형태

애플리케이션에서의 로깅 적용에 앞서 먼저 로그 관리의 유형을 살펴보자. 여러 분류 기준이 있겠지만, 크게는 다음과 같이 애플리케이션 로그와 시스템 로그로 구분할 수 있다.

유형	설명
애플리케이션 로그	애플리케이션 내부 동작 과정에서 발생하는 내용에 대한 로깅 • 네트워크 문제로 인한 데이터베이스 연결 실패 • 데이터베이스 내부 처리에서 발생한 문제 • 서버 시작/종료 관련 메시지 • 기타 서버 프로그램 동작 중 발생한 문제 등
시스템로그	사용자나 데이터와 관련된 내용에 대한 로깅 • 사용자 로그인 히스토리 • 웹의 경우 특정 페이지 요청 기록 • 사용자 권한 관련 • 특정 메서드 호출 및 매개변수 정보 • 처리 데이터 확인용 메시지 등

물론 이러한 유형 이외에 다른 분류 방식을 사용할 수도 있게지만, 보편적인 분류로는 큰 문제가 없을 것이다.

실제 애플리케이션이나 시스템에 로깅을 적용하기 위한 방법으로는 `System.out.println()`을 이용한 원초적인 방법부터 로깅 전용 프레임워크를 이용한 방법에 이르기까지 다양한 구현 형태가 있다. 권장되는 방법은 로깅 전용 프레임워크를 이용하는 것으로 대표적인 로깅 구현 방법은 다음과 같다. 이렇듯 프레임워크를 이용하면 시스템 운영 과정에서 로그 메시지들의 관리 방법을 유연하게 변경할 수 있으므로 유지보수가 용이하다는 장점이 있다.

형태	설명
프로그래머가 임의로 구현한 로깅	프로그래머가 임의로 로깅 정책을 수립하고, 파일 입출력 기반의 로깅 클래스 구현 등을 통한 처리 방법이다. 비교적 손쉽게 구현할 수 있으나, 표준화가 되지 않았기 때문에 유지보수에 문제가 있다.
JCL(Jakarta Commons Logging)을 이용한 로깅	웹애플리케이션뿐만 아니라 일반 애플리케이션까지 범용으로 이용할 수 있는 로깅 프레임워크다. 여러 로깅 API들을 단일화된 접근 방식으로 사용할 수 있도록 해주는 것으로, 보통의 경우 내부적으로는 LOG4J, JDK Avalon 등의 로깅 API를 사용하면서 애플리케이션에는 JCL을 이용해 프로그래밍하는 형태다.
Log4J를 이용한 로깅	LOG4J는 가장 대표적인 로깅 프레임워크로 알려져있다. 비교적 쉽게 이용할 수 있으며 다양한 기능을 제공한다. LOG4J 단독으로 사용될 수 있지만, commons-logging, SLF4J 등과 연계해서 많이 사용
SLF4(Simple Logging Framework)를 이용한 로깅	가장 최근에 나온 로깅 프레임워크이며, 기존 로깅 API들의 문제점들을 보완한 솔루션으로 기존 로깅 프레임워크와의 호환성과 함께 성능적인 측면의 장점으로 인해 최근의 프로젝트들에서는 SLF4J를 많이 사용하는 추세이다.

프로그래머가 직접 로깅 API를 만들 수도 있으나 호환성 측면에서 그리 권장되지 않으며 로깅 프레임워크를 사용하면 여러 로깅 API를 필요에 따라 선택할 수 있으며 애플리케이션에서는 단일 프레임워크를 사용하는 형태로 구현해 로깅 API에 대한 종속성을 없앨 수 있다. JCL, SLF4J, LOG4J 등이 대표적이지만 최근에는 SLF4J와 Log4J를 결합한 형태를 가장 널리 사용한다.

이어서 웹 애플리케이션에서의 로깅 구현 형태를 살펴보자. 단순히 로깅 프레임워크를 사용하는 것도 중요하지만 전체 시스템 관점에서 로그를 처리하기 위한 구조를 설계하는 것 역시 매우 중요한 부분이다. 웹 애플리케이션은 컨테이너에서 동작하기 때문에 애플리케이션뿐만 아니라 컨테이너의 로깅 시스템을 활용할 수도 있다.

형태	설명
서블릿 컨테이너를 이용한 로깅	웹 애플리케이션에서 대표적으로 이용할 수 있는 로깅의 구현 형태다. 톰캣과 같은 컨테이너에서 제공하는 API를 이용해 로깅을 하는 방식으로 톰캣 로그 파일을 이용하게 된다. 시스템 로깅의 처리에는 적합하지만, 애플리케이션 로깅을 처리하기에는 관리적인 문제가 있다.
필터 혹은 리스너를 이용한 로깅	웹 애플리케이션의 특징적인 로깅 구현 형태이며 필터의 특성상 적용 대상을 선택할 수 있으며 request, response 정보를 가로챌 수 있기 때문에 기존 애플리케이션을 수정하지 않고 추가적으로 필요한 로깅을 수행할 수 있다. 리스너의 경우에도 리스너 특징을 이용해 웹 애플리케이션 전반에 걸쳐 필요한 로깅을 수행할 수 있다.
AOP를 이용한 로깅	AOP는 Aspect Oriented Programming의 약자로, 로깅에 국한된 것은 아니며 현재 운영 중인 애플리케이션 코드를 수정하지 않고 기능을 수정하거나 추가할 수 있는 새로운 프로그래밍 기법이다. 로깅 자체가 정형화된 코드 형태와 구조이므로 전체 코드에 대해 체계적인 접근으로 생각해볼 만한 방법이며 스프링 프레임워크 등을 사영하면 더욱 쉽게 이용할 수 있다.

다음으로 로깅 API에서 로그를 남길 때 사용하는 로그 레벨의 종류를 살펴본다. 처리하고자 하는 로그의 수준과 목적에 따라 다음과 같이 6가지 레벨을 지정해서 로그 처리를 한다. 로그 레벨은 로깅 API 및 프레임워크 설정에 중요한 요소이므로 잘 알아두도록 한다.

레벨	처리 메서드
FATAL	가장 심각한 오류를 말한다. 상태 콘솔에 즉시 출력한다.
ERROR	일반적인 오류를 말한다. 상태 콘솔에 즉시 출력한다.
WARN	오류는 아니지만 주의를 요구하는 경우를 말한다. 더 이상 쓸모없는 API 사용 등이 여기에 해당
INFO	런타임 시 관심 있는 이벤트를 말한다. 콘솔상에 즉시 출력한다.
DEBUG	시스템 흐름과 관련된 상세 정보를 말한다. 로그 파일로만 출력한다.
TRACE	가장 상세한 형태의 정보를 말한다. 로그 파일로만 출력한다.

이처럼 로그 레벨을 두는 이유는 주로 발생하는 로그 메시지들을 그룹화하여 처리함으로써 일괄적인 로그 정책을 적용하거나 관리적 차원에서 구분이 용이하도록 하는데 있다.

1.3 SLF4J

SLF4J는 Simple Logging Framework의 자바 버전으로, 애플리케이션에서 로깅 객체를 사용할 수 있는 기본적인 구조만을 제공하고 실제 로깅 처리는 다른 구현체를 쓸 수 있도록 해주는 범용 프레임워크이다.

www.slf4j.org에서 최신 버전을 다운받을 수 있고, 관련 문서를 참조할 수 있다. 홈페이지 초기 화면에는 다양한 오픈소스 프로젝트들이 나열되어있으므로 참고하면 더욱 기초를 탄탄하게 다질 수도 있다.

SLF4J는 기본적으로 라이브러리 형태로 제공되고 실제 로깅 처리를 위해 필요한 jar파일을 추가적으로 프로젝트 빌드 경로에 추가해주어야 한다.

더욱 구체적으로 살펴보면 기본 API파일인 `slf4j-api-1.7.5.jar`은 반드시 필요하고, 실제 로깅 구현체로 사용할 `slf4j-simple-1.7.5.jar` 혹은 `slf4j-log4j12-1.7.5.jar` 등의 라이브러리는 필요에 따라 추가로 설치해야 한다. `slf4j-simple-1.7.5.jar`은 간단한 콘솔 출력에 필요한 것이고, `slf4j-log4j-1.7.5.jar`은 LOG4J 연동에 필요한 것이다. 이때 SLF4J와 연결되는 두 개의 로깅 구현체는 동시에 사용할 수 없으므로 주의한다.

`log4j`는 logging.apache.org/log4j/1.2/download.html에서 다운로드 할 수 있다. 그리고 각각의 jar파일을 [WEB-INF/lib]폴더에 복사하기만 하면 된다. 최종 설치 파일 목록은 다음과 같다.

- SLF4J 라이브러리 : `slf4j-api-1.7.5.jar`
- 로깅 구현체 중 택일(반드시 둘 중 하나만 복사되어 있어야 함): `slf4j-simple-1.7.5.jar` / `slf4j-log4j12-1.7.5.jar`
- Log4K 라이브러리 : `log4j-1.2.17.jar`

이와 같이 `slf4j`는 `api`와 구현이 분리된 구조로 필요에 따라 `simple` 구현체 혹은 `log4j` 구현체 등을 바꿔가며 사용할 수 있다. 또한 `log4j`를 이용할 경우 프로그램 코드 수정 없이 `log4j`의 설정 파일만으로 다양한 기능을 활용할 수 있다.

`log4j` 설정과 관련해서는 뒤에서 자세히 살펴보고 여기서 중요한 것은 SLF4J의 로깅 구현체는 동시에 존재할 수 없고 `simple`혹은 `log4j` 둘중 하나의 파일만 복사해야된다는점을 명심해야한다.

이제 라이브러리 설치에 대한 설명이 끝났으므로, 실제 프로그램에서 어떻게 로그를 남길 수 있는지 알아보자. 기본적으로 로깅 API에 대한 불러오기(import)와 로깅을 위한 로거 객체 확보, 로깅 처리 세단계로 구성한다.

Step 1	로깅 API에 대한 패키지 불러오기
1	import org.slf4j.Logger;
2	import org.slf4j.LoggerFactory;

Step 2	로깅을 위한 Logger 객체 확보
1	Logger logger = LoggerFactory.getLogger(Class class);
2	Logger logger = LoggerFactory.getLogger(String name);

로그 처리를 위한 로거 객체는 LoggerFactory.getLogger()메서드를 이용해서 구하면 된다. 이렇게 구해진 로거 객체는 아래와 같이 활용할 수 있다. 별도의 애플리케이션 프레임워크 기반으로 프로젝트를 진행하는 경우 로거 객체에 대한 참조를 개별 애플리케이션이 아닌 중앙에서 일괄 제공할 수도 있다.

Step 3	로깅 처리
1	log.info(Object message);
2	log.debug("Alert msg is {}", msg);

첫번째 문장은 Logger클래스 참조를 통해 로그 레벨과 이름이 동일한 메서드를 통해 로그 처리를 수행하는 예고, 두 번째 문장은 로그 메시지를 템플릿 형태로 출력하는 방법이다. 기존 commons-logging 등에서는 log.debug("Alert msg is"+msg)와 같이 문자열 결합으로 로그를 처리했으나 성능상의 문제 등 비효율적인 문제가 있어 SLF4J에서는 매개변수 방식으로 처리할 수 있게 됐다. 즉 {} 위치에 msg가 출력되는 형태이다.

예제	LogTest.java
1	package Log;
2	import org.slf4j.Logger;
3	import org.slf4j.LoggerFactory;
4	public class LogTest {
5	public static void main(String[] args) {
6	String msg = "Hello Log!!";
7	Logger logger = LoggerFactory.getLogger(LogTest.class);
8	logger.info("test log");
9	logger.warn("test log : {}",msg);
10	}
11	}

1.4 | Log4j

이제 실제 로깅 구현체인 Log4j를 살펴보자. Log4j는 현재 가장 널리 사용되고 있는 오픈소스 로깅 프레임워크로, 다양한 설정을 통해 체계적인 로그 관리를 지원한다. Log4j는 자체 API를 제공하기 때문에 해당 API를 이용해 프로그래밍할 수 있지만, 호환성을 유지하려면 가급적 SLF4J API를 이용해서 프로그래밍할 것을 권장한다.

- Log4j 설치하기

Log4j는 현재 Apache Logging Service 프로젝트로 통합되었으며, 언어 중립적인 로깅 프레임워크로 개발되고 있다. 현재 가장 널리 사용되고 있는 버전은 1.2.x이며 기존 Log4j의 성능과 기능을 크게 개선한 log4j 2버전이 베타 서비스 중에 있다. 그리고 호환성을 유지하며 통합 운영되는 프로젝트는 다음과 같다.

- Log4Cxx(C++)
- Log4j(1.x, 2-beta)
- Log4Net
- Log4PHP

Log4j는 <http://logging.apache.org/log4j>에서 최신 버전과 관련 자료를 다운로드할 수 있으며, 예제 제공 사이트에 있는 log4j-1.2.17.jar파일을 다운로드해서 [WEB-INF\lib]폴더에 설치한다. 또한 SLF4J와 연동하려면 slf4j-1.7.5.jar파일의 복사도 필요하다. 앞서 말한바와 같이 slf4j-simple-1.7.5.jar파일을 복사해 두었다면 먼저 삭제 후 복사한다.

-Log4j 설정하기

Log4j를 사용하려면 먼저 log4j 설정 파일을 만들어야 한다. log4j는 단순히 로그를 출력하는 역할만 하는 것이 아니라 로그 메시지의 형식, 저장 방식등을 다양하게 지정할 수 있다. 설정방식은 두가지인데, 자바의 속성 파일이나 xml 형식을 선택해 사용할 수 있다. xml 설정 방식이기 나중엔 나중엔 좀 복잡하기는 하지만 더욱 구조적이고 매개변수 파일보다 다양하게 설정할 수 있으므로 여기서는 xml형식의 설정 방법을 기준으로 살펴보자.

설정 파일은 톱캣에 직접 배포한 웹 애플리케이션의 경우 [WEB-INF\classes]폴더에 저장해야 하지만, 이클립스의 경우에는 [src] 루트 폴더에 넣어두면 되고 나중에 내보내면 자동으로 웹 애플리케이션 아카이브 구조로 정리되어 배포된다. 따라서 굳이 설치되는 위치에 신경 쓸 필요가 없다. 여기서는 [src]폴더에서 작성한다.

요소

내용

<root> </root>	root 로거를 정의한다 기본 로그 설정이라고 보면 된다 • level : FATAL, ERROR, WARN, INFO, DEBUG • appender-ref : 해당 appender의 세부 설정을 할 때 사용할 이름
<logger> </logger>	관리할 로거를 정의한다. 패키지별 로그 레벨에 따른 로거 이름을 설정한다 • level : FATAL, ERROR, WARN, INFO, DEBUG • appender-ref : 해당 appender의 세부 설정을 할 때 사용할 이름
<appender> </appender>	root에서 설정한 appender 이름에 대한 실제적인 appender와 레이아웃등을 지정한다 • appender 클래스 종류 참조 • layout클래스 참조

<root>와 <logger>는 기본적으로 같은 개념으로 로깅 설정에 필요한 그룹으로 이해하면 된다. <root>는 <logger>를 포함해 공통적으로 기본 적용할 로그 설정을 하는 부분이고 <logger>는 특정 로그 설정을 하는 영역이 된다.

appender는 로그 메시지의 출력 대상을 지정하는 것으로 아래의 표와 같이 다양한 종류가 있으며 생성할 로그 파일의 성격에 따라 하나, 혹은 appender 여러 개를 설정할 수 있다. 예를 들어, 특정 애플리케이션에 대해서는 기본적으로 콘솔로 출력하고 추가적으로 파일로도 출력하도록 설정할 수 있으며, 다른 애플리케이션에서는 윈도우 시스템 로그로 기록하면서 이메일로도 전달할 수 있다.

appender와 layout에는 이외 여러 하위 키 값이 있으나 각각의 종류에 따라 설정 항목이 다르므로, 이와 관련한 자세한 내용은 아래의 표와 log4j 관련 문서를 참조하도록 한다.

종류	내용
ConsoleAppender	콘솔에 로그메시지를 출력한다.
FileAppender	파일에 로그 메시지를 기록한다
RollingFileAppender	파일에 로그 메시지를 기록하고, 파일 크기가 일정 수준 이상이 되면 다른 이름으로 저장하고 새롭게 기록한다.
DailyRolling FileAppender	파일에 로그 메시지를 기록하고, 하루 단위로 로그 파일을 변경해서 저장한다.
SMTPAppender	로그 메시지를 이메일로 전송한다.
NTEventLogAppender	NT 시스템 이벤트 로그로 메시지를 전송한다. 윈도우 NT/XP/2003 서버등에 해당한다.

이외에도 JDBCAppender, SocketAppender, TelnetAppender 등 다양한 서드 파티 appender가 있으며 마음에 드는 appender가 없을 경우 custom appender를 직접 구현해서 사용할 수 도 있다.

layout은 로그 메시지의 출력 포맷과 관련된 것으로, 몇 가지 기본 유형이 있다. 일반적으로 사용자 정의 패턴에 따라 메시지 출력을 자유롭게 설정할 수 있는 PatternLayout을 많이 사용한다.

종류	내용
DateLayout	로그 메시지를 날짜 중심으로 간단하게 기록한다
HTMLLayout	로그 메시지를 HTML 형식으로 기록한다
PatternLayout	로그 메시지를 사용자 지정 패턴에 따라 기록한다.
SimpleLayout	레벨-메시지 형식의 가장 간단한 로그를 기록한다
XMLLayout	로그 메시지를 XML 형식으로 기록한다.

이 중에서 가장 많이 쓰이는 PatternLayout은 다음과 같은 패턴 구문을 사용해서 메시지 형식으로 지정한다.

형식	내용
%c	카테고리를 출력한다
%p	로깅 레벨을 출력한다
%m	로그 내용이다
%d	로깅 이벤트가 발생한 시간이다. yyyy-MM-dd, HH:mm:ss 등 시간 형식을 사용한다
%t	로그 이벤트가 발생한 스레드의 이름을 출력한다
%%	%기호를 출력한다
%n	개행 문자(new line)다.
%C	클래스 이름이다.
%F	로깅이 발생한 파일 이름이다.
%l	로깅이 발생한 호출자 정보다
%L	로깅이 발생한 행 수다.
%M	로깅이 발생한 메서드 이름이다
%r	애플리케이션 시작 이후부터 로깅이 발생한 시점의 시간이다
%x	로깅이 발생한 스레드와 관련된 NDC를 출력한다
%X	로깅이 발생한 스레드와 관련된 MDC를 출력한다.

이러한 로깅 설정은 구문이나 규칙만으로는 이해하기 힘들고 실제 적용되는 예제들을 통해 기능을 익히고 내게 맞는 형태로 수정해서 사용하는 것이 좋다.

SLF4J, Log4j 연동하기 Section 2

2.1 실습개요

간단한 설정을 통해 SLF4J와 Log4j를 연동한 로깅 예제를 살펴보자. 실습은 xml형식의 log4j 설정을 사용하고, 설정 내용은 INFO 레벨의 경우 콘솔로 출력하고, WARN 메시지의 경우 파일에 로그 내용을 기록하는 형식으로 구성한다. 로깅 테스트는 앞서 SLF4J 테스트에서 사용한 LogTest.java를 그대로 이용한다.

- 실습파일

log4j.xml : src , LogTest.java : src\jspbook\ch14

slf4j와 log4j 라이브러리 파일들은 미리 [WEB-INF\lib] 폴더에 복사된 상태여야한다.

2.2 log4j.xml 작성

다음은 INFO 레벨과 WARN 혹은 그 상위 레벨의 로그를 기록하기 위한 log4j.xml 파일로, 콘솔에는 비교적 간단히 출력하고 파일에 더욱 상세한 내용을 기록하는 실습이다.

log4j.xml

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE log4j:configuration PUBLIC "-//APACHE//DTD LOG4J 1.2//EN" "http://logging.apache.org/log4j/1.2/
  apidocs/org/apache/log4j/xml/doc-files/log4j.dtd">
3 <log4j:configuration xmlns:log4j="http://jakarta.apache.org/log4j/">
4     <appender name="stdout" class="org.apache.log4j.ConsoleAppender">
5         <layout class="org.apache.log4j.PatternLayout">
6             <param name="ConversionPattern"
7                 value="%d{HH:mm:ss} [%c{1}] [%p] %m %n" />
8         </layout>
9     </appender>
10    <appender name="dailyout" class="org.apache.log4j.DailyRollingFileAppender">
11        <param name="file" value="c:/tmp/dailyout.log"/>
12        <param name="Append" value="true"/>
13        <param name="DatePattern" value=".'yyMMdd"/>
14        <layout class="org.apache.log4j.PatternLayout">
15            <param name="ConversionPattern" value="%t> [%d{yyyy-MM-dd HH:mm:ss}] [%c{1}] [%L] [%p] %m %n"/>
16        </layout>
17    </appender>
18    <root>
19        <level value="INFO" />
20        <appender-ref ref="stdout" />
21        <appender-ref ref="dailyout" />
22    </root>
23 </log4j:configuration>

```

위치	내용
01행~02행 (xml 파일 선언부)	XML파일임을 알리는 부분이며 log4j.dtd와 네임 스페이스에 대한 부분을 지정하고있다.
05행~19행 (appender 설정)	appender는 실제 로그를 처리하는 모듈이며 여기서 appender를 두개 지정했는데, stdout의 경우 ConsoleAppender에 대한 설정으로 패턴 레이아웃을 이용해 메시지를 콘솔에 출력하라는 설정이다. 그리고 dailyout은 RollingFileAppender에 대한 설정으로 매일 갱신되는 파일 기반의 appender 이다. appender가 요구하는 속성들이 있으며 [c:/tmp]폴더에 dailyout.log라는 파일을 매일 생성하며, 지난 파일은 연월일 순으로 날짜가 붙어 저장되게 된다. 역시 패턴 레이아웃을 사용하고 stdout보다는 많은 양의 정보를 설정하고 있다.
20행 ~24행 (logger 설정)	여기서는 root로거만 설정하고 있으며 INFO레벨 이상의 메시지에 대해서 stdout,dailyout 두 appender를 이용해서 로깅하도록 지정하고 있다.

2.3 실행결과 확인

xml파일이 준비되었으면 LogTest 클래스를 실행해보도록 한다. 콘솔상의 실행 결과는 이전과 유사하지만 패턴 레이아웃 적용으로 더욱 많은 정보와 체계적인 구성을 확인할 수 있으며, 실행 결과는 콘솔과 파일 두곳으로 출력된다.

콘솔의 결과는 아래와 같이 출력된다.

	콘솔 출력 결과
1	[21:47:44] [LogTest] [INFO] test log
2	[21:47:44] [LogTest] [WARN] test log : Hello Log!!

다음으로 c:/temp/daily.log 파일을 메모장으로 열어보면 다음과 같이 더욱 상세한 로그가 출력된 것을 확인할 수 있다.

	메모장 출력결과
1	main> [2022-05023 32:10:46] [LogTest] [11] [INFO] test log
2	main> [2022-05023 32:10:46] [LogTest] [12] [WARN] test log : Hello Log!!
3	main> [2022-05023 32:13:47] [LogTest] [11] [INFO] test log
4	main> [2022-05023 32:13:47] [LogTest] [12] [WARN] test log : Hello Log!!

이처럼 SLF4J와 Log4j를 이용하면 애플리케이션을 수정하지 않고 로그 처리를 상황에 맞게 다양하게 운영할 수 있다.

웹 애플리케이션 로깅 기법

Section 3

앞서 일반적 프로그램에서의 로깅을 위한 설정을 배웠다면 이번에는 웹 애플리케이션에서의 로깅 기법을 배워보도록 한다. 기본적인 로깅 기법은 `slf4j`와 `log4j`를 연동해서 활용하는 것이지만, 웹 애플리케이션의 특징을 활용하면 더욱 유용한 로깅 구현을 할 수 있다.

3.1 서블릿 컨테이너 제공 로깅

웹 애플리케이션 로깅은 기본적으로 사용할 수 있는 로깅 방법으로, 주로 시스템 로깅을 처리하기 위해 이용한다. 웹 애플리케이션의 경우 컨테이너와 밀접한 관계가 있으므로 서블릿 컨테이너 제공 로깅을 잘 알고 있으면 유용하게 활용할 수 있다.

컨테이너마다 로그 파일 이름이나 위치 등이 다르며, 여기에서는 톰캣을 기준으로 설명한다.

컨테이너 제공 로깅은 `javax.servlet.ServletContext` 클래스에서 제공하는 API를 이용한다.

메서드	내용
<code>public void log(String msg)</code>	컨테이너 로깅 시스템을 통해 문자열 형태의 내용을 출력한다. 기본 톰캣의 경우 <code>localhost.yyy-MM-dd.log</code> 형태의 파일에 기록된다. 이클립스의 경우 콘솔 메시지만 출력된다. 로그 레벨은 INFO로 취급된다.
<code>public void log (String msg, Throwable throwable)</code>	<code>Throwable</code> 클래스는 모든 자바 오류와 예외의 슈퍼 클래스다. 메시지와 함께 특정 예외 처리 메시지를 함께 로깅할 수 있다. 로그 레벨은 SEVERE(심각)로 취급된다.

`ServletContext` 클래스는 `jsp`의 경우 `application` 내장객체로 참조할 수 있으면, 서블릿 등에서는 `getServletContext()` 메서드를 통해 참조할 수 있으므로 이를 이용하면 된다.

컨테이너에서 발생하는 로그에 대한 처리는 컨테이너 종류마다 다르며, 톰캣의 경우 다음의 디렉터리에서 로그 파일을 관리한다.

실행환경	내용
톰캣 단독 실행	톰캣 설치 디렉터리\logs
이클립스에서 톰캣 실행	<code>workspace\.metadata\.plugins\org.eclipse.wst.server.core\tmp0\logs</code>

톰캣 단독 실행의 경우 여러가지 로그 파일을 날짜별로 관리하고 있으며, `ServletContext`를 이용한 로그 메시지는 `localhost.yyyy-MM-dd.log` 형태의 파일에 기록된다.

이클립스에서 실행한 톰캣의 경우에는 서비스 운영이 아닌 개발 테스트의 성격이 강하므로, 효율성을 위해 별도의 로그 파일로 기록하지 않고 콘솔 메시지 형태로만 출력한다. 필요하다면 콘솔 메시지를 파일화할 수 있으며, `server.xml`을 수정해서 로그 파일을 추가할 수도 있다.

3.2 필터를 이용한 로깅

웹 애플리케이션 요청에 대해 request 및 response를 가로채고, 체인 형식의 적용을 통해 동적인 애플리케이션 운영을 가능하게하는 서블릿 스펙이다. 이러한 필터의 속성을 이용해 시스템 로그와 애플리케이션 로그의 중간적인 정보를 제공하는 로깅을 구현할 수 있다. 즉 단순한 애플리케이션 하나에 대한 로그가 아닌 전체적인 서비스 차원에서의 정보를 체계적으로 처리할 수 있다.

필터를 이용한 로깅은 시스템의 사용자 행위를 추적하는데 유용하다. 예를 들어 포털의 경우 특정 사용자가 로그인 후 사용하는 게시판이나 관심을 가지는 게시물, 광고의 트래킹 등 사용자 취향 분석등에도 활용될 수 있다.

필터를 이용한 로깅 자체는 특별한 것은 없으며, 일반적인 필터의 개발 구조다. 다만 로깅 처리를 하려면 다음과 같은 부분을 고려해서 설계해야한다.

항목	설명
로그파일이름	보통 web.xml에 초기화 매개변수로 파일 이름을 등록한 후 참조한다.
로그처리	init() 메서드 및 destroy() 메서드에서 처리한다. 로깅 프레임워크를 이용한다.
필터매핑	필터의 특징은 특정 URL 패턴에 따라 여러 필터를 산별적으로 적용할 수 있다는 점이다. 로깅 정책에 따른 적절한 매핑 설정이 중요하다.

3.3 리스너를 이용한 로깅

필터와 마찬가지로 리스너 역시 표준 서블릿 스펙이므로, 모든 웹 애플리케이션에서 호환 가능한 로깅을 구현할 수 있다.

리스너의 경우 컨테이너의 특정 이벤트에 기반을 둔 로깅 구현에 적합하며, 필터와 동일하게 시스템 로그와 애플리케이션 로그의 중간적인 정보를 처리한다.

필터와 마찬가지로 전체 시스템 설계 관점에서 고려되어야 하는 것으로 이러한 로깅 정책 수립은 어느 정도의 시스템 관리 및 운영 경험이 요구된다. 다음은 리스너를 이용한 로깅 구현을 할 때 고려한 것들이다.

항목	설명
로그파일이름	보통 web.xml에 초기화 매개변수로 파일 이름을 등록한 후 참조한다
로그처리	자카르타 Commons 및 Log4j등의 프레임워크를 이용하며 다음과 같은 리스너 생명주기 메서드에서 처리한다. contextInitialized() 및 contextDestroyed() 메서드 sessionCreated() 및 sessionDestroyed() 메서드

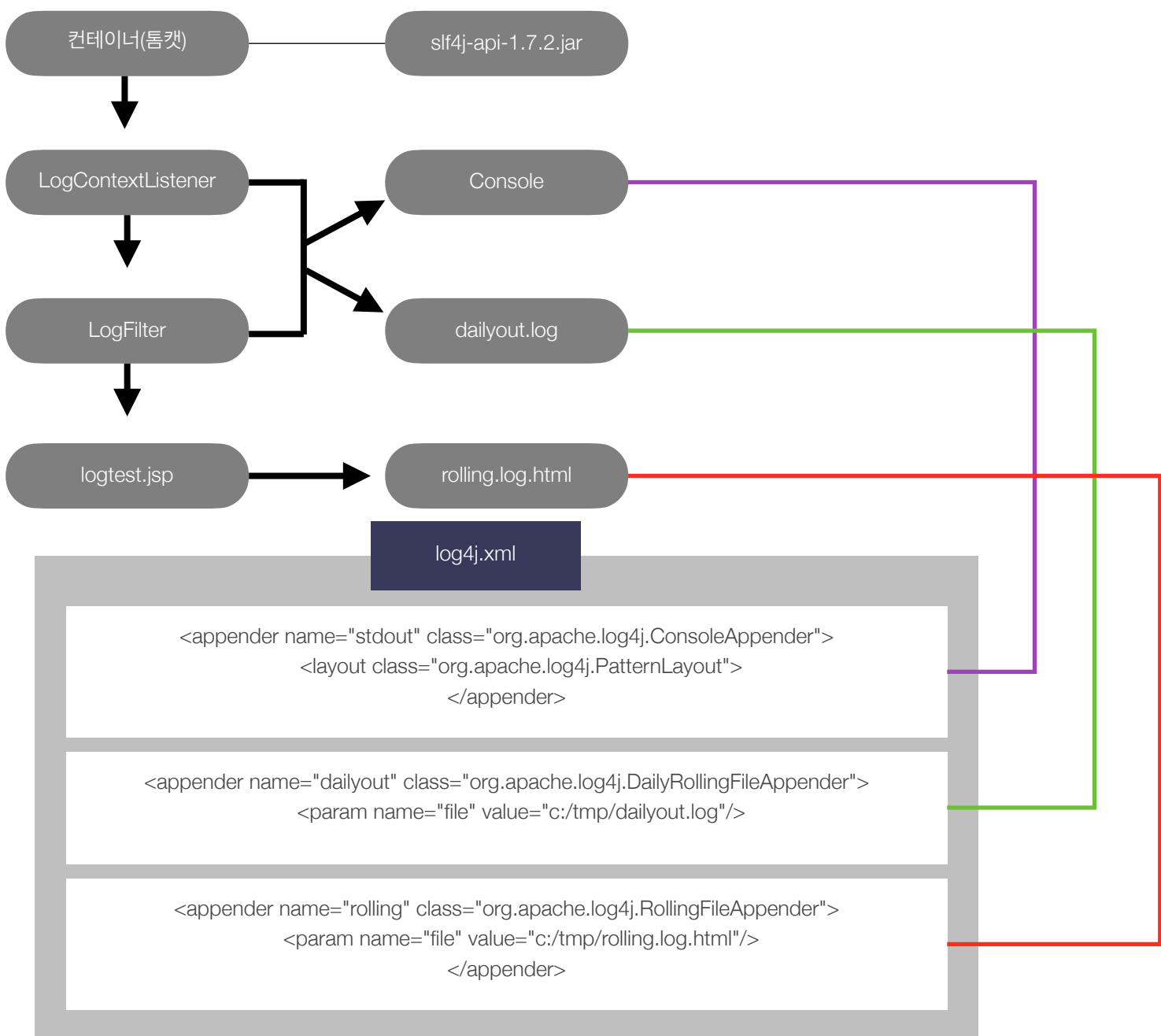
시스템 전반에 걸친 로깅 정책구현

Section 4

4.1 실습 개요

지금까지 웹 애플리케이션에서의 다양한 로깅 기법들을 살펴보았다. 이제부터는 지금까지의 내용을 정리하는 차원에서 웹 애플리케이션에 대한 전반적인 로깅 정책을 수립하고, 필터, 리스너등을 이용한 웹 애플리케이션 로깅 구현을 실습하자.

이번절에서 다루는 웹 애플리케이션 종합 예제는 시스템 전반에 걸쳐 로깅 정책을 수립하고 그에 따라 다양한 구성요소를 통해 로그를 처리한다. 전반적인 로깅 예제 시나리오는 그림과 같다.



로깅 설정은 log4j.xml에서 이루어지고 시나리오에 따른 구현 내용은 다음과 같다.

기본 로깅 구현은 SLF4J와 Log4j를 사용한다.

모든 웹 애플리케이션 기본 로그는 SimpleLayout을 사용해 콘솔로 출력하도록 설정한다.

LogContextListener는 톰캣 시작과 종료시 콘솔과 DailyRollingFileAppender에 따라 매일 생성되는 파일에 로그를 처리한다.

LogFilter는 모든 JSP파일에서 발생하는 로그를 처리하며 콘솔 및 일정 크기 이상에서 파일을 갱신하는 RollingFileAppender에서 HTMLLayout을 이용해 로그를 관리한다.

4.2 log4j.xml 파일 설정

여기서는 파일을 새로 만들지 않고 앞서 만들었던 log4j.xml 파일에 몇 가지 새로운 설정을 추가하도록 한다.

앞선 시나리오에 따라 HTMLLayout을 사용하는 RollingFileAppender를 추가하고 root 로거 이외에 프로젝트.패키지의 자바 클래스에서 출력하는 로그 처리를 하려는 로거와 jsp들에서 출력하는 로그 처리를 위한 로거를 추가한다.

log4j.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE log4j:configuration PUBLIC "-//APACHE//DTD LOG4J 1.2//EN" "http://logging.apache.org/log4j/1.2/
3  apidocs/org/apache/log4j/xml/doc-files/log4j.dtd">
4  <log4j:configuration xmlns:log4j="http://jakarta.apache.org/log4j/">
5      <appender name="stdout" class="org.apache.log4j.ConsoleAppender">
6          <layout class="org.apache.log4j.PatternLayout">
7              <param name="ConversionPattern" value="%d{HH:mm:ss} [%c{1}] [%p] %m %n" />
8          </layout>
9      </appender>
10     <appender name="dailyout" class="org.apache.log4j.DailyRollingFileAppender">
11         <param name="file" value="c:/tmp/dailyout.log"/>
12         <param name="Append" value="true"/>
13         <param name="DatePattern" value="'.yyMMdd'"/>
14         <layout class="org.apache.log4j.PatternLayout">
15             <param name="ConversionPattern" value="%t> [%d{yyyy-MM-dd HH:mm:ss}] [%c{1}] [%L] [%p] %m %n"/>
16         </layout>
17     </appender>
18     <appender name="rolling" class="org.apache.log4j.RollingFileAppender">
19         <param name="file" value="c:/tmp/rolling.log.html"/>
20         <param name="Append" value="true"/>
21         <param name="MaxFileSize" value="10kb"/>
22         <param name="MaxBackupIndex" value="1"/>
23         <layout class="org.apache.log4j.HTMLLayout" />
24     </appender>
25     <logger name="jspbook.ch14">
26         <level value="INFO" />
27         <appender-ref ref="dailyout" />
28     </logger>

```

log4j.xml

```

29 <logger name="org.apache.jsp.ch14">
30   <level value="WARN" />
31   <appender-ref ref="rolling" />
32 </logger>
33 <root>
34   <level value="INFO" />
35   <appender-ref ref="stdout" />
36   <appender-ref ref="dailyout" />
37 </root>
38 </log4j:configuration>

```

다른 부분은 앞서 이미 설명했고 일반적인 사항이라 크게 문제가 없을 것이다. 다만 웹 애플리케이션 전체가 아니라 대상을 한정하기 위해 특정 패키지를 지정하고 있다.

위치

내용

28~36행
 (logger 추가)

먼저 jspbook.ch14는 해당 패키지의 클래스에서 출력되는 로그들을 처리하려는 로거다. INFO 레벨 이상에 대해 stdout과 dailyout appender를 이용해 로그를 처리한다. 두번째로 org.apache.jsp.ch14는 jsp에서 발생하는 로그 처리를 위한 로거다. 이 경우 특이한 점이 하나 있다. jsp에 대한 로그를 추적하려면 톰캣에 의해 변환되는 자바 패키지인 org.apache.jsp를 사용해야하는데, 여기서는 [ch14] 폴더를 사용하므로 org.apache.jsp.ch14가 해당 jsp파일에 대한 패키지 이름이 된다는 점이다.

20~26행
 (appender 추가)

추가된 appender는 로그 파일이 특정 크기가 되면 자동으로 새로운 파일을 생성하는 RollingFileAppender으로, 레이아웃은 HTMLLayout으로 지정했다. 이렇게 하면 로그 파일의 내용이 HTML규격으로 생성되므로 웹 브라우저를 통해 더욱 깔끔한 로그 기록을 조회할 수 있다. 여기서는 테스트를 하려고 로그 파일 크기를 10kb로 설정하였다.

4.3 리스너 로그 구현

이번에는 ServletContextListener를 통해 웹 애플리케이션 시작하고 종료할 때 로그를 생성하도록 한다. 기본 생명주기 메서드에 간단한 로그를 추가했다. 참고로 여기에서 만든 리스너가 등록되면 프로젝트 전체에 적용되는 것이므로 만약 불필요할 경우에는 테스트 이후 동작하지 않도록 주석처리해야 한다.

LogContextListener.java

```
1 package jspbook.ch14;
2
3 import javax.servlet.ServletContextEvent;
4 import javax.servlet.ServletContextListener;
5 import javax.servlet.annotation.WebListener;
6
7 import org.slf4j.*;
8
9 @WebListener
10 public class LogContextListener implements ServletContextListener {
11     // 로그 처리를 위한 로거 객체
12     Logger log;
13
14     // 리스너 종료시 로그 메시지 출력
15     public void contextDestroyed(ServletContextEvent arg0) {
16         log.info("LogContextListener stop");
17     }
18
19     // 리스너 초기화시 로그 메시지 출력
20     public void contextInitialized(ServletContextEvent arg0) {
21         log = LoggerFactory.getLogger(this.getClass());
22         log.info("LogContextListener start");
23     }
24 }
```

웹 애플리케이션을 시작하고종료할 때 간단한 로그 메시지를 출력하도록 되어 있다. 로깅은 slf4j를 이용했다.

4.4 필터 로그 구현

이번에는 패키지의 모든 jsp에 적용할 필터를 만든다. 필터에서는 필터 생명주기 메서드에서 각각 로그를 남기게 된다.

LogFilter.java

```

1 package jspbook.ch14;
2
3 import java.io.IOException;
4
5 import javax.servlet.Filter;
6 import javax.servlet.FilterChain;
7 import javax.servlet.FilterConfig;
8 import javax.servlet.ServletException;
9 import javax.servlet.ServletRequest;
10 import javax.servlet.ServletResponse;
11 import javax.servlet.annotation.WebFilter;
12
13 import org.slf4j.*;
14
15 //ch14 폴더 요청에 대해서만 필터 적용
16 @WebFilter("/ch14/*")
17 public class LogFilter implements Filter {
18     // 로그 처리를 위한 로거 객체
19     Logger log;
20
21     // 필터 종료시 로그메시지
22     public void destroy() {
23         log.info("LogFilter stop");
24     }
25
26     // 필터 실행시 로그메시지
27     public void doFilter(ServletRequest arg0, ServletResponse arg1,
28         FilterChain arg2) throws IOException, ServletException {
29         log.debug("LogFilter run"+arg0.getRemoteAddr());
30         arg2.doFilter(arg0, arg1);
31     }
32
33     // 필터 초기화시 로그메시지
34     public void init(FilterConfig arg0) throws ServletException {
35         log = LoggerFactory.getLogger(this.getClass());
36         log.info("LogFilter start");
37     }
38 }

```

URL 매핑은 해당 패키지 폴더 아래에 있는 모든 요청에 대해서 처리하도록 되어있다. 각각의 라이프 사이클 메서드에서 로그를 남기고 있다. 한 가지 추가적으로 로그 레벨 설정에 따른 동작을 확인하려고 doFilter()메서드에서 debug()메서드를 이용한 로그를 처리하였으며 이는 xml설정에 따라 출력되지 않는다(로그 레벨상 debug 레벨의 출력이 설정되어 있지 않기 때문이다.)

4.5 logtest.jsp 작성

마지막 단계로 jsp파일에서 slf4j를 이용해 로그를 작성하는 방법을 살펴보자. 기본적으로 jsp가 뷰 역할을 수행하기 때문에 jsp자체에서 로그를 남기는 경우는 드물다. 또한 jsp등의 웹 리소스의 요청에 대한 로깅은 톰캣 자체적으로 수행하고 있기 때문에 jsp에 개발자가 직접 로깅 코드를 작성하는 경우는 거의 없다고 볼 수 있다. 여기서는 단순하게 테스트를 목적으로 jsp에서 스크립트릿을 이용한 로깅을 해볼 것이다.

LogFilter.java

```

1  <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" import="java.io.*,
2  org.slf4j.*"%>
3
4  <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
5  <html>
6  <head>
7  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
8  <title>ch14 : log 테스트</title>
9  </head>
10 <body>
11 <div align="center">
12 <H2>ch14:logtest.jsp</H2>
13 <HR>
14 로그 종합 테스트로 화면상에는 아무것도 출력되지 않지만 콘솔 메시지 및 log 디렉토리에 파일로 로그가 생성 됩니다.
15 <%
16 // 시스템 기본 로그를 통해 처리함.
17 application.log("logtest.jsp:테스트 로그 메시지...",new IOException());
18 Logger log = LoggerFactory.getLogger(this.getClass());
19 log.info("info-jsp 파일에서 처리한 로그");
20 log.warn("warn-jsp 파일에서 처리한 로그");
21 %>
22 </div>
23 </body>
24 </html>

```

위치

내용

01행 (패키지 불러오기)	page 지시어를 이용해 로깅에 필요한 패키지를 불러온다(import)
16~19행 (로그 처리)	application.log는 컨테이너를 통한 로그 출력으로 IOException을 강제로 발생시키고 있다. 다음 slf4j를 이용한 로그의 경우 JSP는 서블릿으로 변환되는 클래스 이름이 컨테이너에 따라 차이가 있을 수 있으므로 this.getClass()메서드를 사용해 로거 객체를 구한 다음 로깅 처리를 한다.

4.6 실행결과 확인

툼캣이 실행된 상태라면 종료 후 logtest.jsp를 실행해보도록 한다. 구성요소가 좀 많기 때문에 중간에 하나라도 실수가 있다면 한 번에 성공하기 어려울 것이다. 만일 오류 메시지를 만난다면 처음부터 차근차근 내용을 점검해보도록 한다.

단, logtest.jsp의 application.log() 메서드에서 IOException을 발생시키므로 IOException메시지가 출력되는 것은 정상이다.

여기서는 해당 패키지 리소스에 대해서만 로깅 설정을 했기 때문에 다른 폴더의 jsp에서 발생하는 오류나 로그는 로그 파일로 기록되지 않는다.

콘솔로그 확인

리스너와 필터의 경우 콘솔에 메시지가 출력된다. 다음과 같은 로그가 나오면 제대로 출력된것이다.

-리스너와 필터에 처리된 로그 : INFO레벨

```
[22:22:22] [LogContextListener] [INFO] LogContextListener start
```

```
[22:22:22] [LogFilter] [INFO] LogFilter start
```

logtest.jsp에서 application 내장객체를 이용해 처리된 로그 : SEVERE 레벨

```
05월 30, 2022 10:44:33 오후 org.apache.catalina.core.ApplicationContext log
```

심각: logtest.jsp: 테스트 로그 메시지...

```
java.io.IOException
```

```
at org.apache.jsp.ch14.logtest_jsp._jspService(logtest_jsp.java:79)
```

```
at org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
```

(중략)

logtest.jsp에서 스크립트릿을 이용해 처리된 로그 : WARN 레벨

```
[22:22:22] [logtest_jsp] [WARN] warn-jsp 파일에서 처리한 로그
```

dailyout.log 파일 로그 확인 : INFO, WARN레벨

dailyout.log 파일에도 설정에 따라 다음과 같이 로그를 확인할 수 있다.

메모장 출력결과	
1	localhost-startStop-1> [2022-05023 32:10:46] [LogContextListener] [20] LogContextListener start
2	http-nio-8080-exec-2> [2022-05023 32:10:46] [logtest_jsp] [138] [WARN] warn-jsp파일에서 처리한 로그

기본 로거에서 INFO 이상을 dailyout.log에 남기기 때문에 jsp의 WARN로그도 함께 기록된 것을 알 수 있다.

rolling.log.html 파일의 로그 확인 : WARN 레벨

HTMLLayout을 사용한 로그 파일로 jsp에 대한 로그만 기록되어있다. html 파일이므로 더블클릭해서 브라우저에서 확인이 가능하다.